CSE 167 (WI 2021) Homework 4 Bonus Credit

This is the specification for the bonus credit for HW4. Note that this extra credit part of HW4 has the same due date as main part of HW4. Late submission with late penalty only applies to the main part of the HW. That is to say, no late submission is allowed for the bonus part.

4.1 Global Illumination

This extra credit is to expand your HW4 into a full global illumination rendering. In particular, not only that the specular reflection is computed through a recursive ray tracing, but also that the diffuse reflection is computed recursively.

> **R** The full explanation of the optics involved for global illumination will be covered in the more advanced graphics course (CSE168). In that more complete exposition, one would first understand the rendering equation, an integral equation for the light intensity, and then introduce a randomized algorithm called the Monte Carlo method to evaluate the solution to the integral equation. We will also cover a bit of those rendering equations as extra topics in Week 9 in this course (CSE167).
>
> However, here, we will take a different route. We directly interpret the final Monte Carlo algorithm as a simulation for light particles (photons). By arguing how the photons should bounce between the objects in the scene, we will arrive at the same algorithm. That is to say, this extra credit don't require you to study the more advanced radiometry to build the program.

4.1.1 Simulating photons' paths from the camera

The idea is that we paint each pixel of the image by the total colored light (in RGB) contributed by all the photons that would travel from every light source to the perspective center through the pixel (see Figure 1).

How would a photon travel in the scene? Instead of simulating their physical trajectory from the light sources to the eye, we trace the trajectory backward from the pixel back to the lights. This latter strategy is more convenient if we only want to collect those rays that pass through the pixel. It is also reminiscent to the recursive mirror reflection in the main part of your HW4.

The light path for a concrete number of bounces

Let us first consider paths with a known number of bounces n. To simulate/sample one of these paths, first shoot a random ray from the eye through the pixel. Unlike the main part of your homework where the ray pass the center of the pixel, here we take the ray through a uniformly distributed random point in the pixel. This is illustrated in Figure 1.

Next, as it intersects with a surface, we toss a coin (50%-50%) to decide whether we want the ray to reflect like a diffuse reflection or a specular reflection. If we go about the diffuse mode, then we shoot the next ray at a random direction (described more concretely below). If we go about the specular reflection, then we shoot the next ray along the mirror reflection direction (like in the main part of HW4).

If we have reached the nth bounce, then we terminate the random process. The color of this last hit is computed using the direct diffuse lighting like in the main part of HW4:

$$\text{Color}(\text{hit}_{\text{last}}) = \sum_{\ell \in \text{Lights}} \begin{pmatrix}\text{visibility}\\ \text{to light}_\ell\end{pmatrix} \frac{L_\ell D}{\text{attenuation}_\ell} \langle \mathbf{l}_\ell, \mathbf{n} \rangle^+ \tag{1}$$

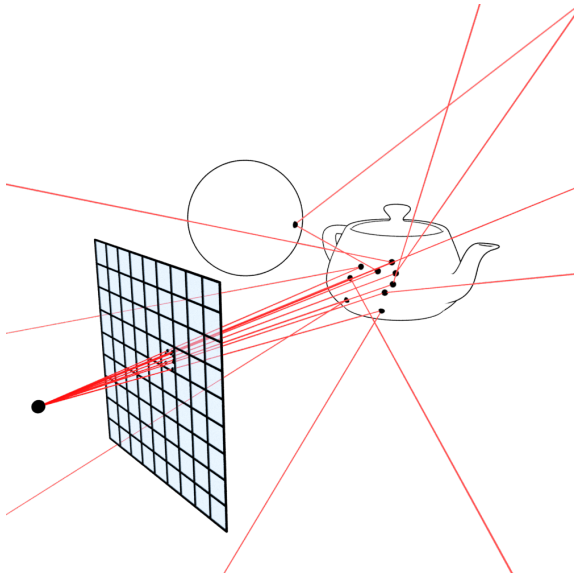

Figure 1 The color of each pixel is the average of all colors of rays (paths of photons) through the pixel that connect the vantage point to the light source. A diffuse material scatters the ray at random direction, while a specular material mirror reflects the ray. A path of n bounces randomly picks the diffuse mode or specular mode of reflecting at every bounce; at the last bounce, shade the final hit by the diffuse lighting model; this amounts to directly connecting the ray to every light sources (with visibility check). The ray color at the pixel (the color to be summed to the pixel) is the final hit color multiplied by the diffuse colors (with Lambert cosine law) and/or the specular colors along the journey.

where D is the diffuse color, and L_ℓ is the light color of the ℓ-th light. Effectively, the last bounce forces the light ray to connect to each visible light source, contributing the color at this last bounce with a Lambert cosine weight.

Finally, the color of this photon path through the pixel (*i.e.* beginning of the path) is the last color multiplied by the material diffuse/specular color at all the intermediate bounces:

$$\text{Color}(\text{RayAtPixel}) = W_1 W_2 \cdots W_{n-1} \text{Color}(\text{hit}_{\text{last}}), \tag{2}$$

where the weight W_i is the diffuse or the specular color of the material at the i-th bounce depending on whether that bounce is of a diffuse mode or a specular mode.

Emissive Surface

If the light is bouncing off from a surface with emission, then we want to add that emission color E_i to the path color. In a recursive formulation,

$$\text{Color}(\text{RayAtPixel}) = \text{Color}(\text{RayBetweenHit}_0\text{AndHit}_1) \tag{3}$$

$$\begin{aligned}\text{Color}(\text{RayBetweenHit}_{i-1}\text{AndHit}_i)& \\ = E_i + W_i \cdot \text{Color}(\text{RayBetweenHit}_i\text{AndHit}_{i+1}),& \quad i = 1, 2, \ldots, n-1\end{aligned} \tag{4}$$

$$\text{Color}(\text{RayBetweenHit}_{n-1}\text{AndHit}_n) = E_n + \text{Color}(\text{hit}_{\text{last}}). \tag{5}$$

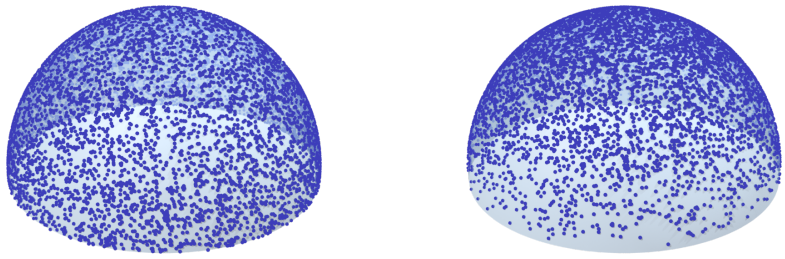

Figure 2 Left: Uniform hemisphere. Right: Cosine-weighted hemisphere.

R Note that we don't add ambient color like in the shading model for HW2 and the main part of HW4. Our full global illumination with recursive lighting replaces the ambient lighting.

Diffuse bounce

Suppose we are bouncing the ray using the diffuse mode; that is, we want to shoot the next ray at a random direction. What should the distribution of the random direction be?

One direct approach is to sample this next ray direction uniformly on the hemisphere (above the surface), like Figure 2 (left). Let $\mathbf{d}$ be this new random ray direction. If one does so, the weight at the bounce is

$$W_i = \underbrace{2\cdot}_{\substack{\text{for the coin}\\ \text{toss for the diffuse/}\\ \text{specular decision}}} D_i \underbrace{\langle \mathbf{d}, \mathbf{n} \rangle}_{\cos \angle^{\mathbf{n}}_{\mathbf{d}}}, \quad \mathbf{d} \text{ is sampled uniformly on the hemisphere.} \tag{6}$$

where the inner product is the Lambert cosine term.

Another approach is to sample the next ray direction $\mathbf{d}$ with a distribution on the hemisphere that is already weighted by the Lambert cosine factor, shown in Figure 2 right. That is, there is a higher probability to shoot a ray near the normal direction than the directions nearly tangent to the surface. In that case, the weight is just the diffuse color

$$W_i = \underbrace{2\cdot}_{\substack{\text{for the coin}\\ \text{toss for the diffuse/}\\ \text{specular decision}}} D_i, \quad \mathbf{d} \text{ is sampled according to the cosine-weighted distribution.} \tag{7}$$

The latter approach will let the final image converge a bit faster than the former uniform sampling.

These hemisphere samplings are obtained by the following mapping formulas. Let s, t be two independent uniform random numbers in the interval $[0, 1]$. Then a uniform hemisphere

sampling of $\mathbf{d}$ is given by

$$u = 2\pi s, \quad v = \sqrt{1-t^2}, \quad \mathbf{d} = \begin{bmatrix} v\cos(u) \\ t \\ v\sin(u) \end{bmatrix}. \tag{8}$$

A cosine weighted hemisphere sampling of $\mathbf{d}$ is given by

$$u = 2\pi s, \quad v = \sqrt{1-t}, \quad \mathbf{d} = \begin{bmatrix} v\cos(u) \\ \sqrt{t} \\ v\sin(u) \end{bmatrix}. \tag{9}$$

Remember to rotate the frame so that the hemisphere is pointing in the normal direction.

4.1.2 Path summation

In the earlier discussion about sampling paths (and their colors) with a fixed number of bounces, we can compute the color contributed by all paths by n bounces. For each pixel (i, j), we take a large number N of random paths and take the average of their resulting colors

$$\text{TotalColorFromPhotonsWith}n\text{Bounces}_{ij} = \frac{1}{N}\sum_{k=1}^{N} \text{Color}(\text{RayAtPixel}_{ij,n,k}). \tag{10}$$

The final pixel color is the sum of the color from all path lengths:

$$\begin{aligned} \text{PixelColor}_{ij} = {} & \text{TotalColorFromPhotonsWithOneBounce}_{ij} \\ & + \text{TotalColorFromPhotonsWithTwoBounces}_{ij} \\ & + \text{TotalColorFromPhotonsWithThreeBounces}_{ij} \\ & + \cdots \end{aligned} \tag{11}$$

In practice, you may truncate this infinite series so that you only compute up to a certain number of bounces (like a maximal recursion depth). In your implementation, you may use this direct truncation method, or the Russian Roulette method.

4.1.3 Russian Roulette

Instead of truncating the path by setting an artificial choice of maximal recursion depth, the Russian Roulette method is a practical method that sums all the terms in the infinite series (11).

First of all, introduce a parameter $0 < \lambda < 1$. Now modify the expression tautologically at first

$$\begin{aligned} \text{PixelColor}_{ij} = {} & \frac{(1-\lambda)\lambda}{(1-\lambda)\lambda}\text{TotalColorFromPhotonsWithOneBounce}_{ij} \\ & + \frac{(1-\lambda)^2\lambda}{(1-\lambda)^2\lambda}\text{TotalColorFromPhotonsWithTwoBounces}_{ij} \\ & + \frac{(1-\lambda)^3\lambda}{(1-\lambda)^3\lambda}\text{TotalColorFromPhotonsWithThreeBounces}_{ij} \\ & + \cdots \end{aligned} \tag{12}$$

Then, instead of picking a fixed path length n and sampling paths with that length, we will let the photon run indefinitely. But, at every bounce, we toss a coin with probability λ to decide

whether we want to terminate the path tracing. In that way, we have a probability $(1-\lambda)^{n-1}\lambda$ to obtain a path of n bounces (to survive $(n-1)$ bounce and to be killed at the nth bounce). By looking at each term of (12), we learn that we should re-weight the resulting color from an n-bounce path by a factor of $\frac{1}{(1-\lambda)^{n-1}\lambda}$. By doing so, the expectation value of the resulting color will unfold into the formula (12), which is the same as what we want (11).

```
ColorSum_ij = 0;
for (i,j) ∈ Image, k = 1,2,...,N do
    Color_ij,k = (0,0,0); TotalWeight_ij,k = (1,1,1); Factor = 1;
    n_k = 1;                                                    ▷ Path length
    Ray_ij,k = RandomRayThruPixel_ij();
    while True do
        hit = Intersect(Ray_ij,k, Scene);
        if hit = ∅ then break;
        terminate = random { 1  probability = λ
                           { 0  probability = (1 − λ)
        if terminate then
            Factor ∗ = λ;
            Color_ij,k + = TotalWeight_ij,k(E_hit + FinalDiffuseLighting_hit);
            break;
        else
            n_k + +;
            Factor ∗ = (1 − λ);
            Color_ij,k + = TotalWeight_ij,k E_hit;
            TotalWeight_ij,k ∗ = W_hit;                         ▷ Specular/Diffuse decision
            Ray_ij,k = bounce into next ray;                    ▷ Specular/Diffuse decision
        end if
    end while
    ColorSum_ij + = (1/Factor) Color_ij,k;
    PixelColor_ij = (1/N) ColorSum_ij.
end for
```

4.2 Specification

This extra credit is open-ended. The minimal requirement is to implement the global illumination, and design your own artistic scene (for example a *Cornell box* (search this keyword online and you will see what that is)). The objects and their materials should be placed to best display the nontrivial results from your global illumination.

The boost the impressiveness of the result, you may implement more types of geometry. You see, the geometry in ray tracing is solely defined by the intersection function. Sometimes defining intersection is much easier than building a polygon mesh; for example the union/intersection/subtraction between solid shapes can be defined logically in terms their ray-object intersection; those boolean shapes are very difficult to obtain if they were triangle meshes.

Submit your rendering and a readme document that summarizes what you have done. Also attach the zip file of all the source code.

A few best renderings will be displayed in the gallery on the course website.